

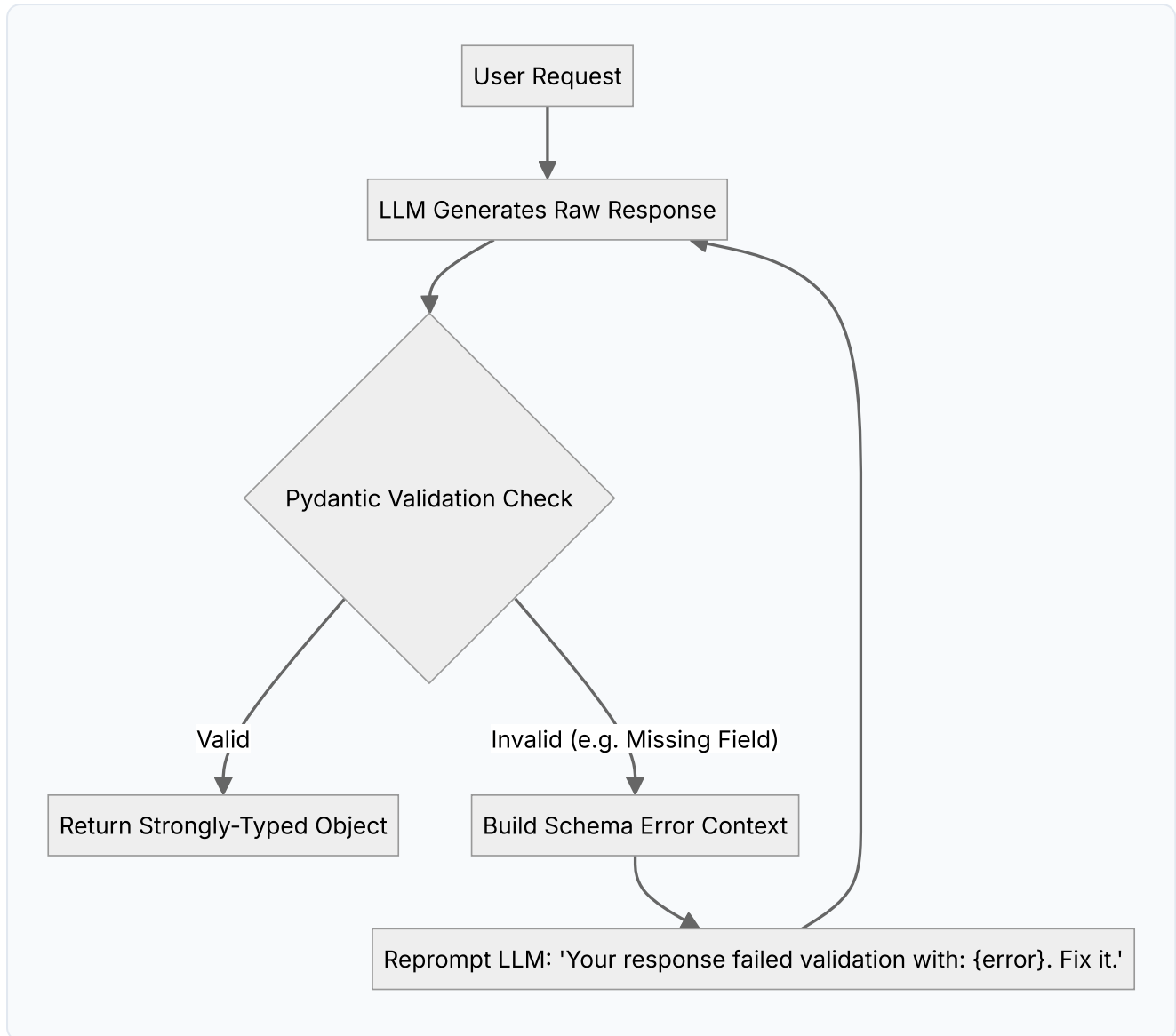
Chapter 10: Production Resilience, Defensive Prompting, & Rate Limits

The Non-Deterministic Nightmare Writing an LLM agent script in a Jupyter Notebook that succeeds once is easy. Running an agent that executes 100,000 workflows a day across enterprise customers without silently failing, hallucinating corrupt JSON, or draining your credit card via runaway recursive loops is an entirely different engineering discipline.

Production Agentic AI engineering is **defensive engineering**. This chapter covers schema healing, token bucket rate limiters with jitter, multi-provider fallback cascades, and circuit breakers.

1. Schema Enforcement & Healing with Pydantic & Instructor

LLMs frequently generate invalid JSON: trailing commas, unescaped quotes, or markdown code fences (````json`) wrapped around payloads.



Self-Healing Extraction Pipeline

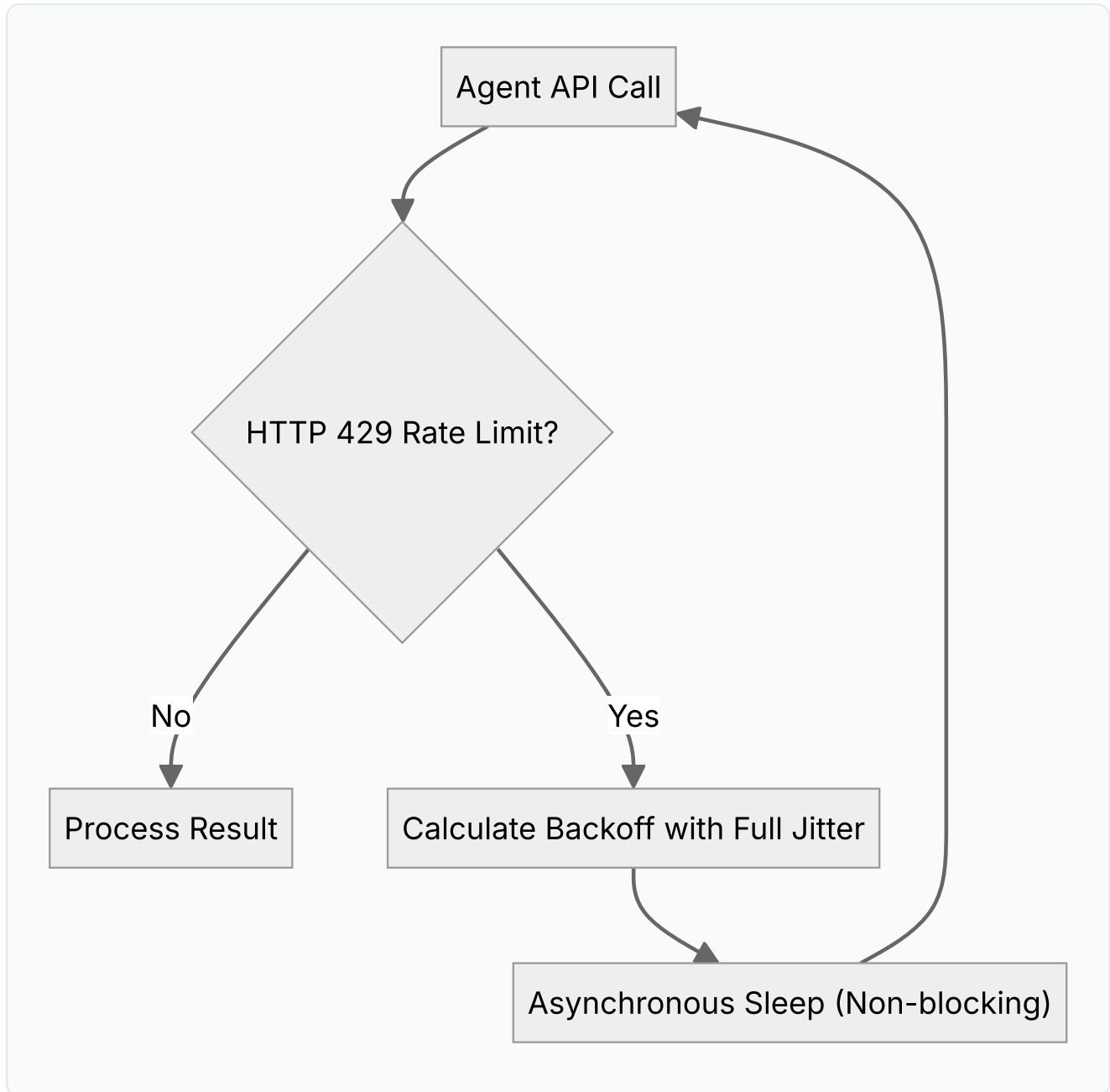
2. Rate Limits, Throttling, & Full Jitter Backoff

When you scale to 50 concurrent agents, OpenAI or Anthropic will immediately return `HTTP 429 Too Many Requests`.

Why Exponential Backoff Without Jitter Causes Avalanches

If 50 agents hit a rate limit at the exact same millisecond and all sleep for $2^1 = 2$ seconds, they will all wake up at the exact same millisecond and collide again!

$$\text{Sleep Time} = \text{random}(0, \min(M, B \times 2^{\text{attempt}}))$$

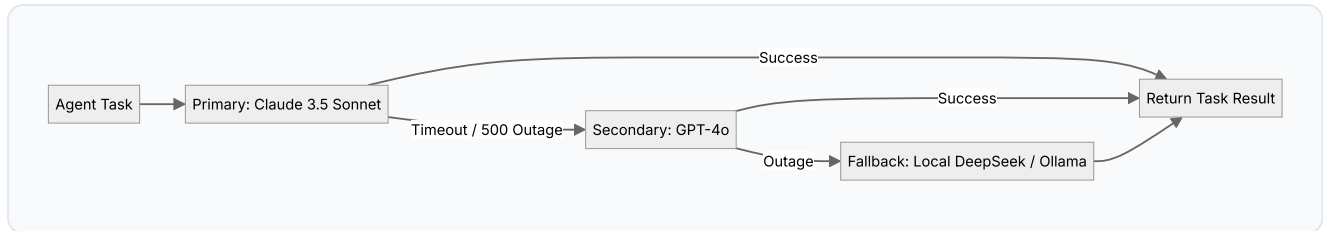


Production Rate Limiter with Full Jitter



3. Multi-Provider Fallback Cascade

Relying on a single LLM provider in production is an existential risk. When OpenAI has an API outage, your business stops. A resilient agent architecture uses a **Cascade**:



Cascade Implementation

4. Runaway Recursive Loop Circuit Breaker

An autonomous agent with tools can easily enter an infinite loop:

- Agent runs `search("weather in London")`
- Tool returns error: `KeyError: temp`
- Agent repeats `search("weather in London")` forever, consuming \$500 in tokens in 20 minutes.

The Strict Budget Token & Step Circuit Breaker

Every production agent loop must wrap its execution inside this circuit breaker!

4. Runnable Model: Circuit Breaker and Retry with Jitter

Model APIs fail in bursts: a rate limit, a timeout, a regional incident. Two small components turn those failures from outages into slowdowns.

Why jitter matters. Without it, every client that failed at the same moment retries at the same moments (1 s, 2 s, 4 s), creating synchronised waves that keep a recovering provider down. Randomising the delay spreads the load.

A decision table for model-API failures

Symptom	Retry?	Action
429 rate limited	Yes, honour <code>Retry-After</code> , with jitter	Queue and smooth traffic; lower concurrency
500 / 503 provider error	Yes, bounded attempts	Breaker, then fail over to a second model or provider
Timeout	Only if the call is idempotent	Shorter timeouts plus hedged requests for latency-critical paths
400 invalid request	No	Fix the request; retrying cannot help
Content filter or refusal	No	Return a safe fallback message and log it
Truncated output (<code>max_tokens</code>)	Maybe	Raise the limit or ask for continuation, not a blind retry

Defensive prompting in one paragraph

Put instructions and untrusted data in separate, clearly delimited places; state the task, the output format and what to do when the input is out of scope; validate the output with code (schema, allowed values) instead of trusting the prompt; and assume any prompt can be overridden by hostile input, so keep real authority in permissions, not wording.

Further Reading

- [AWS: exponential backoff and jitter](#)
- [OWASP Top 10 for LLM applications](#)
- [Martin Fowler: circuit breaker](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What does exponential backoff with jitter avoid?

Synchronised retry storms that overload a recovering service.

▼ 2. Why set timeouts on every LLM and tool call?

To bound latency and free resources when a dependency hangs.

▼ **3.** What is defensive prompting?

Structuring prompts so untrusted content is delimited and treated as data, plus validating outputs; it reduces but does not remove injection risk.

▼ **4.** What is a circuit breaker?

A component that stops calling a failing dependency for a period so it can recover and callers fail fast.